

GUÍA PRÁCTICA | EJERCICIOS INDEPENDIENTES

JavaScript aplicado a un panel de actividades

DOM, eventos, datos, asincronía y almacenamiento local

Material de laboratorio diseñado como una práctica nueva y autocontenida. No requiere utilizar ejercicios, archivos ni proyectos desarrollados en semanas anteriores.

Profesor:	Ing. Jeremy Braseth Elizondo Castro
Correo:	jelizondoc@castrocarazo.ac.cr
Carrera:	Bachillerato en Ingeniería Informática
Modalidad:	Laboratorio guiado + práctica independiente
Entrega:	Repositorio individual de GitHub

En esta guía vas a construir, desde cero, un pequeño panel web para administrar actividades académicas. El propósito es practicar JavaScript con un caso distinto al de las semanas anteriores: no se reutiliza una tarjeta personal, una página previa ni un portafolio construido antes. Todo el trabajo parte de una estructura nueva.

Índice

1. Presentación de la guía	3
1.1. Objetivos de aprendizaje	3
1.2. Requisitos previos	4
2. Preparación del entorno	4
2.1. Crear una carpeta nueva	4
2.2. Base HTML del proyecto	4
2.3. Base CSS del proyecto	5
2.4. Comprobación inicial	6
3. Parte I — Fundamentos de JavaScript	7
3.1. Módulo 1: Variables, tipos de datos y mensajes dinámicos	7
3.1.1. Un poco de teoría	7
3.1.2. Ejercicio guiado 1: configuración del panel	7
3.2. Módulo 2: Objetos, arreglos y funciones	8
3.2.1. Un poco de teoría	8
3.2.2. Ejercicio guiado 2: primera actividad académica	8
3.3. Módulo 3: DOM — mostrar tarjetas desde JavaScript	10
3.3.1. Un poco de teoría	10
3.3.2. Ejercicio guiado 3: renderizar actividades	10
3.4. Módulo 4: Eventos — filtrar y actualizar el panel	11
3.4.1. Un poco de teoría	11
3.4.2. Ejercicio guiado 4: filtros y tema visual	11
4. Parte II — JavaScript moderno	13
4.1. Módulo 5: Destructuring, spread y rest	13
4.1.1. Un poco de teoría	13
4.1.2. Ejercicio guiado 5: mejorar la lectura del código	13
4.2. Módulo 6: Promesas y <code>async/await</code>	15
4.2.1. Un poco de teoría	15
4.2.2. Ejercicio guiado 6: simular consulta de disponibilidad	15
4.3. Módulo 7: Fetch API y JSON	16
4.3.1. Un poco de teoría	16
4.3.2. Ejercicio guiado 7: cargar actividades desde JSON	16
4.4. Módulo 8: LocalStorage — preferencias del panel	18
4.4.1. Un poco de teoría	18
4.4.2. Ejercicio guiado 8: recordar tema y última categoría	19
5. Parte III — Retos independientes	20
5.1. Reto 1: buscador de actividades por palabra clave	20
5.2. Reto 2: registro de participantes con validación	20
5.3. Reto 3: tablero de avisos desde JSON	21

5.4. Reto 4: preferencias de accesibilidad	21
5.5. Reto 5 (avanzado, opcional): consulta pública con API	21
6. Preguntas de reflexión	22
7. Entrega en GitHub	22
8. Rúbrica de evaluación	23
Anexo docente: validación técnica del contenido	24

1 Presentación de la guía

Esta práctica está diseñada para que trabajes JavaScript en un escenario completamente nuevo: un **panel de actividades académicas**. A partir de una base mínima de HTML, CSS y JavaScript, vas a crear datos, mostrarlos en pantalla, responder a eventos del usuario, cargar información desde un archivo JSON y guardar preferencias en el navegador.

La guía ya no depende de ejercicios desarrollados en semanas anteriores. Cada actividad utiliza un contexto propio, con ejemplos nuevos y archivos creados específicamente para esta práctica. La intención es que puedas concentrarte en los conceptos de JavaScript sin arrastrar código heredado de otros laboratorios.

El trabajo se organiza en tres momentos:

- **Parte I — Fundamentos:** variables, tipos de datos, funciones, objetos, arreglos, DOM y eventos.
- **Parte II — JavaScript moderno:** destructuring, spread/rest, promesas, `async/await`, Fetch API, JSON y LocalStorage.
- **Parte III — Retos independientes:** cinco retos nuevos, sin guía paso a paso, para demostrar dominio del tema.

Para saber más

El proyecto de esta guía se llama **Panel de Actividades**. Todos los ejercicios están vinculados a ese caso nuevo. Si tienes archivos de semanas anteriores, no los copies ni los modifiques para esta práctica; inicia una carpeta limpia.

1.1 Objetivos de aprendizaje

Al finalizar esta guía deberías poder:

1. Declarar variables con `let` y `const`, seleccionando la opción adecuada según el uso del dato.
2. Organizar información mediante objetos y arreglos.
3. Escribir funciones tradicionales y funciones flecha para reducir repeticiones.
4. Manipular el DOM para crear contenido dinámico desde JavaScript.
5. Programar eventos de usuario, como clics, cambios en listas desplegables y envíos de formularios.
6. Aplicar destructuring y operadores spread/rest en objetos y arreglos.
7. Simular procesos asíncronos con promesas y resolverlos mediante `async/await`.

8. Cargar datos desde un archivo JSON usando `fetch`.
9. Guardar y recuperar preferencias sencillas con `LocalStorage`.
10. Utilizar las DevTools del navegador para revisar errores, datos y almacenamiento local.

1.2 Requisitos previos

- Visual Studio Code con la extensión Live Server instalada.
- Git configurado y una cuenta activa de GitHub.
- Navegador Google Chrome o Microsoft Edge.
- Una carpeta nueva para este laboratorio.

Atención

Esta guía no solicita abrir ni reutilizar el proyecto de semanas anteriores. El estudiante debe crear una carpeta nueva y trabajar únicamente con los archivos indicados en la sección de preparación.

2 Preparación del entorno

2.1 Crear una carpeta nueva

Crea una carpeta llamada `panel-actividades-js`. Dentro de ella prepara la siguiente estructura:

```
panel-actividades-js/  
  index.html  
  css/  
    estilos.css  
  js/  
    script.js  
  data/  
    actividades.json  
  REFLEXION.md
```

2.2 Base HTML del proyecto

Copia este contenido en `index.html`. Este archivo funciona como punto de partida del laboratorio y no depende de ningún documento anterior:

```
1 <!doctype html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Panel de actividades académicas</title>
7   <link
8     href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
9     rel="stylesheet"
10  >
11   <link rel="stylesheet" href="css/estilos.css">
12 </head>
13 <body>
14   <header class="border-bottom bg-light py-3">
15     <div class="container">
16       <p class="text-muted mb-1">Laboratorio independiente de JavaScript</p>
17       <h1 class="h3 mb-0">Panel de actividades académicas</h1>
18     </div>
19   </header>
20
21   <main class="container my-4">
22     <section id="panel-resumen" class="mb-4">
23       <h2 class="h4">Resumen del panel</h2>
24       <p id="descripcion-panel" class="mb-0"></p>
25     </section>
26
27     <section id="controles" class="mb-4"></section>
28
29     <section id="listado-actividades" class="row g-3"></section>
30   </main>
31
32   <script src="js/script.js"></script>
33 </body>
34 </html>
```

2.3 Base CSS del proyecto

Copia este contenido en `css/estilos.css`:

```
1 body {
2   background-color: #f7fafa;
3 }
4
5 .tarjeta-actividad {
6   border: 1px solid #d9e7e8;
7   border-left: 5px solid #0e4f5a;
```

```
8   border-radius: 12px;
9   background-color: #ffffff;
10  padding: 1rem;
11  height: 100%;
12 }
13
14 .etiqueta-categoria {
15   display: inline-block;
16   border-radius: 999px;
17   background-color: #eef7e6;
18   color: #2f6f25;
19   font-size: 0.85rem;
20   font-weight: 700;
21   padding: 0.25rem 0.65rem;
22 }
23
24 body.tema-oscuro {
25   background-color: #1e1e1e;
26   color: #f5f5f5;
27 }
28
29 body.tema-oscuro header,
30 body.tema-oscuro .tarjeta-actividad {
31   background-color: #252526 !important;
32   color: #f5f5f5;
33   border-color: #3a3a3a;
34 }
```

2.4 Comprobación inicial

Escribe esta línea en `js/script.js`:

```
1 console.log('Panel de actividades cargado correctamente');
```

Abre el proyecto con Live Server. Luego presiona F12, entra a la pestaña **Console** y confirma que el mensaje aparece.

Punto de control

Si el mensaje se muestra en consola, el archivo JavaScript está vinculado correctamente. Si no aparece, revisa la ruta `js/script.js`, guarda todos los archivos y vuelve a abrir el proyecto con Live Server.

3 Parte I — Fundamentos de JavaScript

3.1 Módulo 1: Variables, tipos de datos y mensajes dinámicos

3.1.1 Un poco de teoría

Una variable permite guardar un dato con un nombre claro. En JavaScript moderno se recomienda iniciar con `const` y usar `let` solo cuando el dato tenga que cambiar durante la ejecución.

Los datos básicos que se usarán en este módulo son textos, números y valores booleanos. También vas a utilizar **template literals**, que permiten construir mensajes con variables usando comillas invertidas y la sintaxis `${...}`.

3.1.2 Ejercicio guiado 1: configuración del panel

Paso 1. Borra el mensaje de prueba de `script.js` y escribe los datos base del panel:

```
1 const nombrePanel = 'Panel de Actividades Académicas';
2 const sede = 'Universidad Castro Carazo';
3 const periodo = 'III Cuatrimestre 2026';
4 let totalActividades = 0;
5 const panelActivo = true;
```

Paso 2. Imprime el tipo de dato de cada variable:

```
1 console.log(typeof nombrePanel);
2 console.log(typeof sede);
3 console.log(typeof periodo);
4 console.log(typeof totalActividades);
5 console.log(typeof panelActivo);
```

Paso 3. Construye un mensaje descriptivo usando template literals:

```
1 const descripcion = `${nombrePanel}
2 Sede: ${sede}
3 Periodo: ${periodo}
4 Estado: ${panelActivo ? 'activo' : 'inactivo'}`;
5
6 console.log(descripcion);
```

Paso 4. Muestra ese mensaje en la página:

```
1 document.querySelector('#descripcion-panel').textContent =
2   `${nombrePanel} | ${sede} | ${periodo}`;
```


Punto de control

Recarga la página y revisa dos cosas: la consola debe mostrar los tipos de datos y el resumen debe aparecer dentro de la sección “Resumen del panel”. Después responde: ¿por qué `totalActividades` se declaró con `let`?

3.2 Módulo 2: Objetos, arreglos y funciones

3.2.1 Un poco de teoría

Un **objeto** agrupa datos relacionados mediante propiedades. Un **arreglo** permite almacenar varios elementos en una sola estructura. En este laboratorio usarás ambos para representar actividades académicas.

Una función permite reutilizar instrucciones. Puede recibir parámetros y devolver un resultado con `return`.

3.2.2 Ejercicio guiado 2: primera actividad académica

Paso 1. Crea un objeto con los datos de una actividad:

```
1 const actividadPrincipal = {
2   id: 1,
3   titulo: 'Taller de lógica para programación',
4   categoria: 'Programación',
5   fecha: '2026-07-18',
6   cupo: 24,
7   inscritos: 16,
8   facilitador: {
9     nombre: 'Laura Méndez',
10    correo: 'laura.mendez@ejemplo.com'
11  },
12  recursos: ['computadora', 'internet', 'cuaderno']
13 };
```

Paso 2. Accede a algunas propiedades del objeto:

```
1 console.log(actividadPrincipal.titulo);
2 console.log(actividadPrincipal.facilitador.nombre);
3 console.log(actividadPrincipal.recursos[0]);
```

Paso 3. Crea una función que calcule los cupos disponibles:

```
1 const calcularCuposDisponibles = (actividad) => {  
2   return actividad.cupo - actividad.inscritos;  
3 };  
4  
5 console.log(  
6   `Cupos disponibles: ${calcularCuposDisponibles(actividadPrincipal)}`  
7 );
```

Paso 4. Crea un arreglo con varias actividades nuevas:

```
1 const actividades = [  
2   actividadPrincipal,  
3   {  
4     id: 2,  
5     titulo: 'Clínica de consultas sobre HTML semántico',  
6     categoria: 'Web',  
7     fecha: '2026-07-20',  
8     cupo: 20,  
9     inscritos: 8,  
10    facilitador: {  
11      nombre: 'Marco Salas',  
12      correo: 'marco.salas@ejemplo.com'  
13    },  
14    recursos: ['navegador', 'editor de código']  
15  },  
16  {  
17    id: 3,  
18    titulo: 'Mesa de estudio sobre bases de datos',  
19    categoria: 'Bases de datos',  
20    fecha: '2026-07-22',  
21    cupo: 18,  
22    inscritos: 18,  
23    facilitador: {  
24      nombre: 'Natalia Vargas',  
25      correo: 'natalia.vargas@ejemplo.com'  
26    },  
27    recursos: ['SQL Server', 'diagrama entidad-relación']  
28  }  
29 ];  
30  
31 totalActividades = actividades.length;  
32 console.log(`Total de actividades registradas: ${totalActividades}`);
```

Punto de control

Verifica que el arreglo tenga tres actividades y que la función calcule correctamente los cupos disponibles. Luego responde: ¿por qué conviene guardar cada actividad como objeto en lugar de usar variables sueltas?

3.3 Módulo 3: DOM — mostrar tarjetas desde JavaScript

3.3.1 Un poco de teoría

El DOM es la representación del HTML dentro del navegador. JavaScript puede seleccionar elementos, cambiar su contenido y crear nuevas estructuras visuales sin escribirlas directamente en el HTML.

En este ejercicio vas a convertir el arreglo de actividades en tarjetas visibles para el usuario.

3.3.2 Ejercicio guiado 3: renderizar actividades

Paso 1. Selecciona el contenedor donde se mostrarán las tarjetas:

```
1 const contenedorActividades = document.querySelector('#listado-actividades');
```

Paso 2. Crea una función para construir el HTML de una tarjeta:

```
1 const crearTarjetaActividad = (actividad) => {
2   const cuposDisponibles = calcularCuposDisponibles(actividad);
3
4   return `
5     <article class="col-12 col-md-6 col-lg-4">
6       <div class="tarjeta-actividad">
7         <span class="etiqueta-categoria">${actividad.categoria}</span>
8         <h3 class="h5 mt-3">${actividad.titulo}</h3>
9         <p class="mb-1"><strong>Fecha:</strong> ${actividad.fecha}</p>
10        <p class="mb-1"><strong>Facilitador:</strong>
11          ${actividad.facilitador.nombre}
12        </p>
13        <p class="mb-0"><strong>Cupos disponibles:</strong>
14          ${cuposDisponibles}
15        </p>
16      </div>
17    </article>
18  `;
19 };
```

Paso 3. Crea una función para renderizar todo el arreglo:

```
1 const renderizarActividades = (listaActividades) => {
2   contenedorActividades.innerHTML = '';
3
4   listaActividades.forEach((actividad) => {
5     contenedorActividades.innerHTML += crearTarjetaActividad(actividad);
6   });
7 };
8
9 renderizarActividades(actividades);
```

Paso 4. Actualiza también el resumen del panel:

```
1 document.querySelector('#descripcion-panel').textContent =
2   `${nombrePanel} | ${totalActividades} actividades disponibles`;
```

Punto de control

Al recargar, deben aparecer tres tarjetas de actividades. Si no se muestran, revisa la consola. Si aparece `Cannot set properties of null`, confirma que el id del contenedor coincida con `#listado-actividades`.

Tip del desarrollador

En proyectos pequeños, `innerHTML` permite avanzar con rapidez. En proyectos más grandes también se pueden crear elementos con `document.createElement` para tener más control sobre la estructura y los eventos.

3.4 Módulo 4: Eventos — filtrar y actualizar el panel

3.4.1 Un poco de teoría

Un evento ocurre cuando la persona interactúa con la página. Puede ser un clic, la escritura en un campo o el cambio de una lista desplegable. Con `addEventListener` se indica qué función se ejecutará cuando ocurra ese evento.

3.4.2 Ejercicio guiado 4: filtros y tema visual

Paso 1. Agrega controles dinámicos en la sección `#controles`:

```
1 const controles = document.querySelector('#controles');
2
3 controles.innerHTML = `
```

```
4 <div class="row g-2 align-items-end">
5   <div class="col-12 col-md-5">
6     <label for="filtro-categoria" class="form-label">
7       Filtrar por categoría
8     </label>
9     <select id="filtro-categoria" class="form-select">
10      <option value="Todas">Todas</option>
11      <option value="Programación">Programación</option>
12      <option value="Web">Web</option>
13      <option value="Bases de datos">Bases de datos</option>
14    </select>
15  </div>
16
17  <div class="col-12 col-md-4">
18    <button id="btn-tema" class="btn btn-secondary w-100">
19      Cambiar tema
20    </button>
21  </div>
22
23  <div class="col-12 col-md-3">
24    <button id="btn-cupos" class="btn btn-success w-100">
25      Ver con cupo
26    </button>
27  </div>
28 </div>
29 `;
```

Paso 2. Programa el filtro por categoría:

```
1 const filtroCategoria = document.querySelector('#filtro-categoria');
2
3 filtroCategoria.addEventListener('change', () => {
4   const categoriaSeleccionada = filtroCategoria.value;
5
6   if (categoriaSeleccionada === 'Todas') {
7     renderizarActividades(actividades);
8     return;
9   }
10
11   const actividadesFiltradas = actividades.filter((actividad) => {
12     return actividad.categoria === categoriaSeleccionada;
13   });
14
15   renderizarActividades(actividadesFiltradas);
16 });
```

Paso 3. Programa el botón para mostrar únicamente actividades con cupo:

```
1 const btnCupos = document.querySelector('#btn-cupos');
2
3 btnCupos.addEventListener('click', () => {
4   const actividadesConCupo = actividades.filter((actividad) => {
5     return calcularCuposDisponibles(actividad) > 0;
6   });
7
8   renderizarActividades(actividadesConCupo);
9 });
```

Paso 4. Programa el cambio de tema:

```
1 const btnTema = document.querySelector('#btn-tema');
2
3 btnTema.addEventListener('click', () => {
4   document.body.classList.toggle('tema-oscuro');
5 });
```

Punto de control

Prueba el filtro, el botón de actividades con cupo y el cambio de tema. Luego responde: ¿qué método de arreglos permite crear una nueva lista sin modificar el arreglo original?

4 Parte II — JavaScript moderno

4.1 Módulo 5: Destructuring, spread y rest

4.1.1 Un poco de teoría

El destructuring permite extraer datos de objetos o arreglos de forma directa. El operador `...` se usa como **spread** cuando expande datos y como **rest** cuando agrupa varios valores en una sola variable.

4.1.2 Ejercicio guiado 5: mejorar la lectura del código

Paso 1. Aplica destructuring sobre una actividad:

```
1 const {
2   titulo,
3   categoria,
4   facilitador: { nombre: nombreFacilitador }
5 } = actividadPrincipal;
6
```

```
7 console.log(`${titulo} | ${categoria} | ${nombreFacilitador}`);
```

Paso 2. Usa destructuring dentro de una función:

```
1 const generarResumenActividad = ({
2   titulo,
3   categoria,
4   cupo,
5   inscritos
6 }) => {
7   return `${titulo} (${categoria}) - ${cupo - inscritos} cupos disponibles`;
8 };
9
10 console.log(generarResumenActividad(actividadPrincipal));
```

Paso 3. Usa spread para crear una nueva actividad a partir de otra:

```
1 const actividadExtendida = {
2   ...actividadPrincipal,
3   id: 4,
4   titulo: 'Laboratorio de depuración con DevTools',
5   categoria: 'Web',
6   inscritos: 5
7 };
8
9 console.log(actividadExtendida);
```

Paso 4. Usa rest para registrar observaciones variables:

```
1 const agregarObservaciones = (actividad, ...observaciones) => {
2   return {
3     ...actividad,
4     observaciones
5   };
6 };
7
8 const actividadConObservaciones = agregarObservaciones(
9   actividadPrincipal,
10  'Requiere computadora personal',
11  'Se recomienda llegar 10 minutos antes'
12 );
13
14 console.log(actividadConObservaciones.observaciones);
```

Punto de control

Comprueba que la actividad original no se haya modificado. Después identifica en el código dónde ... funciona como spread y dónde funciona como rest.

4.2 Módulo 6: Promesas y async/await

4.2.1 Un poco de teoría

Una promesa representa una operación que terminará más adelante. Puede resolverse correctamente o fallar. Con `async/await` el código asíncrono se lee de forma más ordenada y con `try/catch` se pueden manejar errores.

4.2.2 Ejercicio guiado 6: simular consulta de disponibilidad

Paso 1. Crea una función que simule una consulta lenta:

```
1 const consultarDisponibilidad = (idActividad) => {
2   return new Promise((resolve, reject) => {
3     setTimeout(() => {
4       const actividad = actividades.find((item) => item.id === idActividad);
5
6       if (!actividad) {
7         reject('La actividad solicitada no existe');
8         return;
9       }
10
11      resolve({
12        titulo: actividad.titulo,
13        cuposDisponibles: calcularCuposDisponibles(actividad)
14      });
15    }, 1500);
16  });
17 };
```

Paso 2. Consume la promesa con `async/await`:

```
1 const mostrarDisponibilidad = async () => {
2   console.log('Consultando disponibilidad...');
3
4   try {
5     const resultado = await consultarDisponibilidad(1);
6     console.log(
7       `${resultado.titulo}: ${resultado.cuposDisponibles} cupos disponibles`
8     );
9   }
10 };
```



```
9   } catch (error) {  
10    console.error('No fue posible consultar:', error);  
11  } finally {  
12    console.log('Consulta finalizada');  
13  }  
14 };  
15  
16 mostrarDisponibilidad();
```

Paso 3. Cambia temporalmente el número 1 por 99 para provocar un error. Observa el mensaje del catch y luego vuelve a dejar un identificador existente.

Punto de control

La consola debe mostrar primero el mensaje de consulta, esperar un momento y luego imprimir el resultado o el error. Agrega un `console.log('fin del archivo')` al final del script y observa qué aparece primero.

4.3 Módulo 7: Fetch API y JSON

4.3.1 Un poco de teoría

La Fetch API permite solicitar archivos o datos externos desde JavaScript. En esta guía se usará para leer un archivo JSON local. Este enfoque es útil porque separa los datos de la lógica de programación y evita escribir toda la información directamente dentro del archivo JavaScript.

4.3.2 Ejercicio guiado 7: cargar actividades desde JSON

Paso 1. Crea el archivo `data/actividades.json` con este contenido:

```
[  
  {  
    "id": 101,  
    "titulo": "Club de algoritmos prácticos",  
    "categoria": "Programación",  
    "fecha": "2026-08-03",  
    "cupos": 25,  
    "inscritos": 10,  
    "facilitador": {  
      "nombre": "Andrés Solano",  
      "correo": "andres.solano@ejemplo.com"  
    },  
    "recursos": ["pizarra", "computadora", "editor de código"]  
  },  
]
```

```
{
  "id": 102,
  "titulo": "Taller de prototipado web",
  "categoria": "Web",
  "fecha": "2026-08-05",
  "cupo": 22,
  "inscritos": 14,
  "facilitador": {
    "nombre": "Carolina Rojas",
    "correo": "carolina.rojas@ejemplo.com"
  },
  "recursos": ["navegador", "Figma", "editor de código"]
},
{
  "id": 103,
  "titulo": "Laboratorio de consultas SQL",
  "categoria": "Bases de datos",
  "fecha": "2026-08-09",
  "cupo": 20,
  "inscritos": 7,
  "facilitador": {
    "nombre": "Diego Araya",
    "correo": "diego.araya@ejemplo.com"
  },
  "recursos": ["SQL Server", "datos de prueba"]
}
]
```

Paso 2. Agrega una función para cargar ese archivo:

```
1 const cargarActividadesDesdeJSON = async () => {
2   contenedorActividades.innerHTML = `
3     <div class="col-12">
4       <p class="text-muted">Cargando actividades desde JSON...</p>
5     </div>
6   `;
7
8   try {
9     const respuesta = await fetch('data/actividades.json');
10
11     if (!respuesta.ok) {
12       throw new Error(`Error HTTP: ${respuesta.status}`);
13     }
14
15     const actividadesJSON = await respuesta.json();
16     renderizarActividades(actividadesJSON);
17   } catch (error) {
18     contenedorActividades.innerHTML = `
19       <div class="col-12">
```

```
20     <div class="alert alert-danger">
21         No se pudieron cargar las actividades. Revisa Live Server
22         y la ruta del archivo JSON.
23     </div>
24 </div>
25 `;
26
27 console.error(error);
28 }
29 };
```

Paso 3. Crea un botón para ejecutar la carga:

```
1 controles.innerHTML += `
2     <div class="mt-3">
3         <button id="btn-json" class="btn btn-primary">
4             Cargar actividades desde JSON
5         </button>
6     </div>
7 `;
8
9 document
10 .querySelector('#btn-json')
11 .addEventListener('click', cargarActividadesDesdeJSON);
```

Paso 4. Abre la pestaña **Network** de las DevTools, presiona el botón y revisa la solicitud al archivo `actividades.json`.

Punto de control

Las actividades cargadas desde JSON deben reemplazar las tarjetas iniciales. Si abres `index.html` con doble clic, la lectura del JSON puede fallar. Debes trabajar con Live Server.

4.4 Módulo 8: LocalStorage — preferencias del panel

4.4.1 Un poco de teoría

LocalStorage permite guardar pares clave-valor en el navegador. La información se mantiene aunque se recargue la página o se cierre el navegador. Como solo guarda texto, cuando se almacenan objetos o arreglos se debe usar `JSON.stringify`; al leerlos, se usa `JSON.parse`.

4.4.2 Ejercicio guiado 8: recordar tema y última categoría

En este ejercicio vas a mejorar el código del módulo 4. El objetivo es que el panel recuerde el tema visual y la última categoría seleccionada.

Paso 1. Restaura el tema guardado al cargar la página:

```
1 const temaGuardado = localStorage.getItem('tema-panel');
2
3 if (temaGuardado === 'oscuro') {
4   document.body.classList.add('tema-oscuro');
5 }
```

Paso 2. Modifica el evento del botón de tema para guardar la preferencia:

```
1 btnTema.addEventListener('click', () => {
2   document.body.classList.toggle('tema-oscuro');
3
4   const temaActual = document.body.classList.contains('tema-oscuro')
5     ? 'oscuro'
6     : 'claro';
7
8   localStorage.setItem('tema-panel', temaActual);
9 });
```

Paso 3. Guarda la última categoría seleccionada:

```
1 filtroCategoria.addEventListener('change', () => {
2   const categoriaSeleccionada = filtroCategoria.value;
3   localStorage.setItem('ultima-categoria', categoriaSeleccionada);
4
5   if (categoriaSeleccionada === 'Todas') {
6     renderizarActividades(actividades);
7     return;
8   }
9
10  const actividadesFiltradas = actividades.filter((actividad) => {
11    return actividad.categoria === categoriaSeleccionada;
12  });
13
14  renderizarActividades(actividadesFiltradas);
15 });
```

Paso 4. Restaura la categoría guardada al cargar la página:

```
1 const categoriaGuardada = localStorage.getItem('ultima-categoria');
2
```

```
3 if (categoriaGuardada) {  
4   filtroCategoria.value = categoriaGuardada;  
5   filtroCategoria.dispatchEvent(new Event('change'));  
6 }
```

Punto de control

Selecciona una categoría, cambia el tema, recarga la página y confirma que ambas preferencias se conservan. Luego abre DevTools, entra a **Application** y revisa las claves `tema-panel` y `ultima-categoria`.

Atención

No guardes información sensible en LocalStorage. Este mecanismo es útil para preferencias de interfaz, filtros o datos simples, pero no para contraseñas, tokens, datos bancarios ni información personal delicada.

5 Parte III — Retos independientes

Los siguientes retos son nuevos y no tienen relación con ejercicios de semanas anteriores. Debes resolverlos dentro del proyecto `panel-actividades-js`. Puedes consultar documentación oficial, pero evita copiar soluciones completas. El objetivo es que puedas explicar tus decisiones y defender tu código.

5.1 Reto 1: buscador de actividades por palabra clave

Agrega un campo de búsqueda para localizar actividades por título, categoría o nombre del facilitador. La búsqueda debe ejecutarse mientras el usuario escribe.

Criterios de aceptación:

- La búsqueda no debe distinguir entre mayúsculas y minúsculas.
- Si no hay coincidencias, debe mostrarse un mensaje claro en la página.
- El arreglo original de actividades no debe modificarse.

5.2 Reto 2: registro de participantes con validación

Crea un formulario para registrar una persona interesada en una actividad. El formulario debe solicitar nombre completo, correo, actividad seleccionada y cantidad de acompañantes.

Criterios de aceptación:

- El formulario no debe recargar la página al enviarse.
- El nombre, el correo y la actividad son obligatorios.
- El correo debe contener el carácter arroba.
- La cantidad de acompañantes debe ser un número entre 0 y 3.
- Los errores deben mostrarse dentro de la página, no con `alert`.

5.3 Reto 3: tablero de avisos desde JSON

Crea un archivo `data/avisos.json` con al menos cinco avisos académicos. Cada aviso debe tener título, fecha, prioridad y descripción. Luego usa `fetch` para cargarlos y mostrarlos en una sección nueva del panel.

Criterios de aceptación:

- Los avisos deben vivir únicamente en el archivo JSON.
- La lectura debe usar `async/await` y `try/catch`.
- Los avisos de prioridad alta deben distinguirse visualmente.
- Si ocurre un error, debe mostrarse un mensaje amigable.

5.4 Reto 4: preferencias de accesibilidad

Permite que el usuario active una vista de lectura cómoda. Esta vista debe aumentar ligeramente el tamaño de letra y mejorar el contraste del panel. La preferencia debe guardarse con `LocalStorage`.

Criterios de aceptación:

- La vista cómoda puede activarse y desactivarse con un botón.
- La preferencia se conserva al recargar la página.
- El botón actualiza su texto según el estado activo o inactivo.

5.5 Reto 5 (avanzado, opcional): consulta pública con API

Agrega una sección llamada “Dato técnico del día” que consulte una API pública y muestre un dato breve relacionado con tecnología, programación o cultura digital. Puedes utilizar una API gratuita de datos de prueba como `JSONPlaceholder` o una API pública similar que no requiera autenticación.

Criterios de aceptación:

- La consulta debe realizarse con `fetch`.
- La función debe usar `async/await`.
- Mientras se espera la respuesta, debe mostrarse un texto de carga.
- Si la API no responde, la página debe manejar el error sin romperse.
- El estudiante debe explicar en `REFLEXION.md` qué datos devuelve la API y cómo los leyó desde JavaScript.

Tip del desarrollador

Los retos 1 al 4 corresponden al nivel esperado de esta práctica. El reto 5 es opcional y está pensado para estudiantes que quieran practicar consumo de datos externos.

6 Preguntas de reflexión

Incluye tus respuestas en el archivo `REFLEXION.md`. Responde con tus propias palabras y escribe al menos un párrafo por pregunta:

1. ¿Qué ventaja tiene trabajar con un proyecto nuevo cuando se está aprendiendo un tema técnico?
2. ¿Por qué conviene separar los datos en estructuras como objetos, arreglos o archivos JSON?
3. Explica con un ejemplo propio la diferencia entre una operación síncrona y una operación asíncrona.
4. ¿Qué tipo de información sí guardarías en `LocalStorage` y qué información nunca deberías guardar ahí?
5. ¿Qué parte del panel de actividades te resultó más desafiante: DOM, eventos, JSON, asincronía o `LocalStorage`? Justifica tu respuesta.

7 Entrega en GitHub

1. Revisa que tu repositorio contenga estos archivos: `index.html`, `css/estilos.css`, `js/script.js`, `data/actividades.json`, `data/avisos.json` y `REFLEXION.md`.
2. Desde la terminal de Visual Studio Code, ejecuta la secuencia habitual:

```
git add .
git commit -m "Guía JavaScript: panel de actividades independiente"
git push origin main
```

3. Publica el enlace del repositorio en la asignación correspondiente de Microsoft Teams antes de la fecha límite indicada en clase.
4. Confirma que el repositorio sea público o que el profesor tenga acceso como colaborador.

Atención

El trabajo es individual. Si dos entregas presentan código idéntico sin justificación, se aplicará lo indicado en el acuerdo de aprendizaje del curso. También se revisará el historial de *commits*; se espera observar avances progresivos y no un único *commit* con todo el trabajo al final.

8 Rúbrica de evaluación

Esta guía forma parte de la evaluación del curso: corresponde a un laboratorio guiado y a una práctica independiente. La calificación se distribuye de la siguiente manera:

Criterio	Puntos	Evidencia
Ejercicios guiados 1–4 funcionando	20	Código en el repositorio y demostración en clase.
Ejercicios guiados 5–8 funcionando	20	Código en el repositorio y demostración en clase.
Reto 1: buscador de actividades	10	Cumple los tres criterios de aceptación.
Reto 2: registro de participantes	10	Cumple los cinco criterios de aceptación.
Reto 3: tablero de avisos desde JSON	12	Cumple los cuatro criterios de aceptación.
Reto 4: preferencias de accesibilidad	8	Cumple los tres criterios de aceptación.
Preguntas de reflexión	10	REFLEXION.md completo y con criterio propio.
Buenas prácticas: nombres claros, uso correcto de <code>const/let</code> y ausencia de errores en consola	5	Revisión del código fuente.
Historial de <i>commits</i> progresivo	5	Mínimo cinco <i>commits</i> con mensajes descriptivos.
Total	100	

Para saber más

El reto 5 no se incluye dentro de los 100 puntos de esta rúbrica. Es opcional y puede utilizarse como evidencia adicional de participación o mejora del proyecto, según el criterio indicado por el docente.

Anexo docente: validación técnica del contenido**Validación técnica**

Esta versión fue ajustada para desvincular la guía de ejercicios, páginas o proyectos desarrollados en semanas anteriores. Todos los ejercicios trabajan sobre un caso nuevo: el **Panel de Actividades Académicas**. La base HTML, el CSS, los datos de prueba, los retos y las preguntas de reflexión fueron reemplazados para que el documento sea autocontenido.

Contenido revisado**Resultado de la revisión**

Dependencia con semanas anteriores	Se eliminaron referencias a proyectos previos, páginas personales, portafolios o estructuras trabajadas antes.
Variables, funciones y objetos	Los ejemplos usan datos nuevos relacionados con actividades académicas.
DOM y eventos	Las tarjetas, filtros, botones y secciones se generan dentro de un proyecto nuevo.
Fetch API y JSON	La carga de datos se realiza desde archivos JSON creados para esta guía.
LocalStorage	Las preferencias guardadas corresponden al panel actual: tema visual y última categoría seleccionada.
Retos independientes	Los retos fueron sustituidos por actividades nuevas: buscador, registro de participantes, tablero de avisos y accesibilidad.

Buen trabajo. Al completar esta guía habrás construido un proyecto nuevo y funcional para practicar JavaScript moderno con datos, eventos, asincronía y almacenamiento local.